

Поиск частых наборов алгоритмом Apriori

Экспериментальное исследование baskets.csv

11 сентября 2026 г.

1. Формулировка задания

Разработать программу поиска всех непустых частых наборов объектов с помощью Apriori или его модификации. Входные параметры: файл транзакций, порог поддержки и порядок результатов: по убыванию поддержки либо лексикографически. Для каждого набора вывести состав и поддержку.

На фиксированном наборе baskets.csv провести эксперименты с порогами 1%, 3%, 5%, 10% и 15%. Визуализировать время поиска и количество частых наборов каждой длины. Подготовить PDF с исходной постановкой, ссылкой на материалы и интерпретацией результатов.

2. Репозиторий и материалы

Открыть локальный каталог [Git-репозитория AD](#).

Онлайн-публикация ожидает входа в GitHub. Эта локальная ссылка работает только на исходном компьютере; перед сдачей отчёта её следует заменить адресом опубликованного репозитория.

В каталоге находятся `apriori.py`, `experiments.py`, `build_report.py`, `tests/`, исходный `data/baskets.csv`, полные списки наборов и измерения в `results/`, рисунки в `figures/` и этот отчёт в `output/pdf/`. Команды запуска приведены в `README.md`.

3. Данные и алгоритм

В файле 7,501 транзакция и 115 разных товаров. Каждая строка - отдельная корзина; первая строка также является транзакцией. Разделитель - запятая, кодировка исходного файла - Windows-1251. Пустых корзин нет. Средняя длина после удаления повторов составляет 3.91 товара, максимальная - 20.

Пробелы по краям названий удаляются, пустые поля игнорируются. 39 лишних повторных вхождений товаров внутри корзин удалены: поддержка отражает присутствие товара, а не число купленных единиц. Одинаковые корзины в разных строках сохраняются. Названия товаров не объединяются по смыслу.

Для набора X абсолютная поддержка $\text{count}(X)$ равна числу корзин, содержащих все товары X . Относительная поддержка $\text{support}(X) = \text{count}(X) / N$. Набор частый, если $\text{count}(X) \geq \text{ceil}(s \times N)$, где s - заданный порог. Для точного округления используется Decimal; например, 1% от 7 501 требует не менее 76 корзин.

Используется свойство Apriori: все подмножества частого набора тоже частые [1]. Сначала отбираются одиночные товары. Затем частые наборы длины $k-1$ с общим префиксом объединяются в кандидаты длины k ; кандидаты с нечастым подмножеством длины $k-1$ отбрасываются. После подсчёта поддержки остаются частые кандидаты. Поиск прекращается при пустом очередном уровне.

Модификация подсчёта: каждому товару сопоставляется целое число с битом для каждой содержащей его транзакции. Маска кандидата получается пересечением масок двух родительских наборов (побитовое AND), а поддержка - методом `int.bit_count()`. Сохраняется поуровневая генерация Apriori; готовые библиотеки поиска частых наборов не используются.

Число кандидатов в худшем случае экспоненциально по числу товаров. Битовые пересечения обрабатывают маски из N бит; память зависит от числа масок текущего и следующего уровней. Поэтому быстрый результат на этом файле не гарантирует такую же скорость на плотных данных.

Выход и проверка корректности

JSON содержит `items`, `length`, `count` и `support` для каждого набора. При сортировке `support` сначала идут большие `count`, при равенстве - лексикографический порядок. Режим `lex` сравнивает отсортированные кортежи названий по Unicode, без языковой локали.

Пройдены 4 теста, включая 60 сравнений со всеми подмножествами на случайных малых данных, точную границу порога, обработку CSV и сортировки. На `baskets.csv` результаты всех пяти порогов независимо сверены горизонтальным перебором комбинаций длины 1-3; Apriori также проверил кандидатов длины 4 и не нашёл частых наборов.

4. Постановка экспериментов

Среда измерений: Python 3.12.14, macOS-26.6.2-arm64-arm-64bit. Для каждого порога выполнен один прогрев и 15 измерений. Порядок 75 запусков перемешан с seed=42. Время измерено `time.perf_counter_ns()`; показаны медиана и диапазон минимум-максимум.

CSV читается один раз до измерений. Каждый запуск заново строит битовые маски и выполняет полный поиск. Чтение файла, сортировка, экспорт и построение графиков не входят в измеренное время. Это время ядра поиска, а не полное время команды. Фоновые процессы не изолировались; диапазон не является доверительным интервалом.

Порог	Мин. count	Медиана, мс	Мин.-макс., мс	Наборов
1%	76	7.966	7.661-8.462	261
3%	226	6.167	5.918-6.375	55
5%	376	5.908	5.657-6.191	28
10%	751	5.711	5.490-6.056	7
15%	1126	5.733	5.476-6.279	5

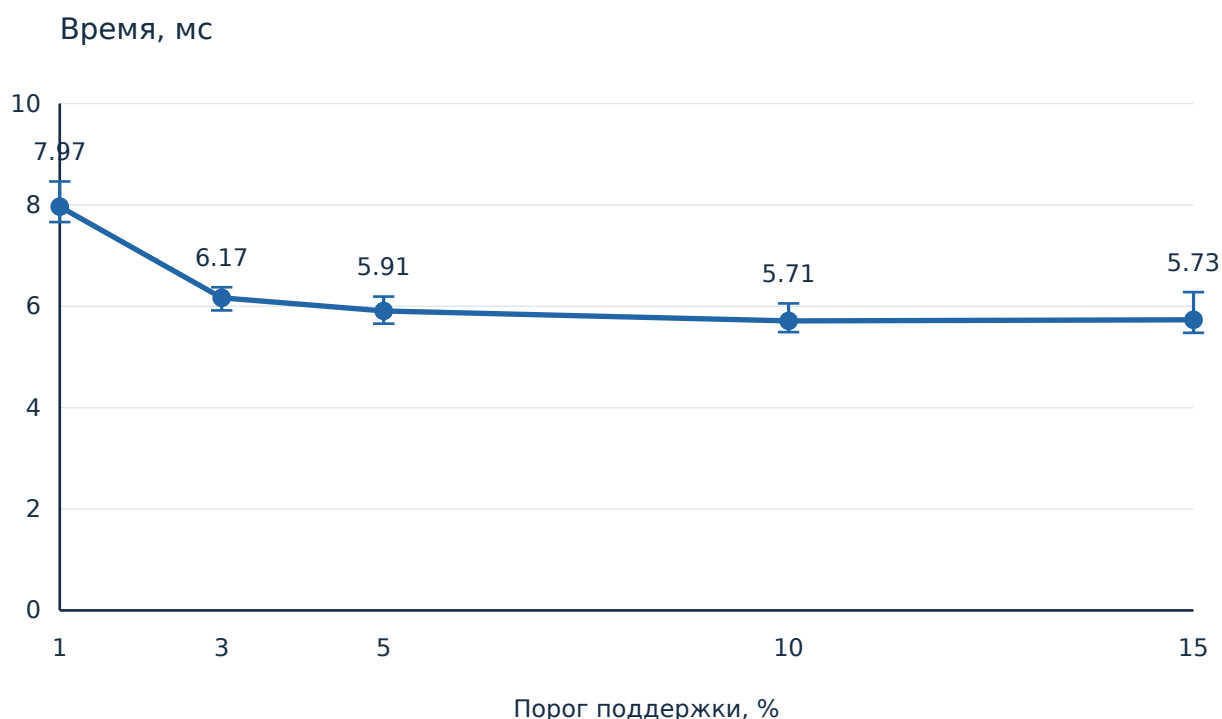


Рисунок 1. Зависимость времени поиска от порога поддержки. Точки - медианы 15 запусков; вертикальные отрезки - минимум и максимум.

При повышении порога с 1% до 15% медиана снизилась с 7.97 до 5.73 мс (примерно в 1.39 раза). Сокращается число кандидатов, но построение исходных масок выполняется при каждом пороге. При 10% и 15% время почти одинаково; небольшое обратное изменение медианы укладывается в наблюдаемый разброс.

5. Количество и длина частых наборов



Рисунок 2. Количество частых наборов длины 1, 2 и 3 при каждом пороге. Подписи 0 обозначают отсутствие наборов соответствующей длины. Для длин 4 и более значения равны нулю при всех порогах.

Порог	1 товар	2 товара	3 товара	Всего
1%	74	170	17	261
3%	36	19	0	55
5%	25	3	0	28
10%	7	0	0	7
15%	5	0	0	5

При 1% преобладают пары: 170 из 261 набора. Также найдены 17 троек. При 3% тройки исчезают, при 5% остаются лишь три пары. При 10% и 15% встречаются только одиночные товары. Чем больше товаров требуется одновременно, тем меньше или равно число подходящих корзин, поэтому длинные наборы быстрее перестают проходить порог.

Для одного и того же файла множество результатов при большем пороге вложено в множество результатов при меньшем. Количество результатов каждой длины поэтому не может увеличиваться. В отличие от числа наборов, измеренное время не обязано строго убывать из-за накладных расходов и шума измерения.

При 1% после отсека проверяются 2 701 пара, 507 троек и 2 четвёрки. При 15% проверяется только 10 пар, ни одна не проходит порог. Это объясняет снижение вычислительной нагрузки. Наборы большей длины отсутствуют уже при минимальном исследованном пороге.

6. Интерпретация и выводы

Набор товаров	Корзин	Поддержка
минеральная вода	1788	23.837%
макароны, минеральная вода	459	6.119%
минеральная вода, шоколад	395	5.266%
минеральная вода, яйца	382	5.093%
говяжий фарш, макароны, минеральная вода	129	1.720%

Минеральная вода встречается в 1 788 корзинах (23,837%). Самая частая пара - макароны и минеральная вода: 459 корзин (6,119%). Поэтому она сохраняется при 5%, но не при 10%. Самая частая тройка - говяжий фарш, макароны и минеральная вода: 129 корзин (1,720%); она проходит порог 1%, но не 3%.

Порог 1% даёт подробное описание совместных покупок ценой более длинного списка результатов и большего времени. Порог 5% выделяет наиболее распространённые пары, а 10–15% оставляет только массовые одиночные товары. Для изучения сочетаний товаров на этом файле пороги 1–5% информативнее; окончательный выбор зависит от цели анализа.

Поддержка характеризует распространённость сочетания, но сама по себе не доказывает зависимость товаров или причинное влияние. В работе не вычислялись правила ассоциации, confidence и lift. Нет сведений о периоде наблюдения и составе покупателей, поэтому результаты описывают только предоставленные 7 501 корзину.

Реализован параметризуемый поиск со всеми требуемыми вариантами сортировки. Эксперимент показал сокращение числа частых наборов с 261 до 5 при повышении порога с 1% до 15%. Воспроизводимость обеспечивают исходный файл, код, тесты, полные результаты и все 75 измерений времени.

Источники и контроль данных

[1] Agrawal R., Srikant R. Fast Algorithms for Mining Association Rules in Large Databases. VLDB, 1994, pp. 487–499. [Текст статьи](#).

Источник данных: baskets.csv, предоставленный пользователем; исходный файл сохранён без изменения байтов. SHA-256:
049a1cb8e869cfa56a4e258e6b5efb1870a536e963deb350da4487667c3c7890

Все числовые результаты и рисунки построены по локальным экспериментам этой работы. Измерения времени зависят от машины и текущей нагрузки.